API REFERENCE

VC

VIRTUAL CONTROL

VMWARE CLOUD FOUNDATION SOLUTIONS

NSX 9 Transport Node Profile Schema Reference

Manager API endpoint, required fields, ID format gotchas (VDS UUID with spaces, manager-API UUIDs vs policy display names), and a verified-working POST body. Built from a 2026-05-06 NSX rebuild that took TNP creation from 4 failures to one success.

NSX 9 | TNP | TRANSPORT NODE PROFILE | MANAGER API | POLICY API | VDS UUID

VCF 9.0 VMware Cloud Foundation

NSX 9 Transport Node Profile — Schema Reference

Prepared by: Virtual Control LLC **Date:** May 6, 2026 **Document Version:** 1.0 **Classification:** Internal — API Reference **Scope:** VCF 9.0.1 nested lab, NSX Manager 9.0.1.0 build 24952114

---

1. Executive Summary

> **Where commands run in this schema reference.** TNP creation runs from:
>
> - **Workstation (curl in Bash, OR Invoke-RestMethod in PowerShell)** — NSX Manager API at `https://<nsx-manager-fqdn>/api/v1/...`. Auth: HTTP Basic with `admin` user.
> - **NSX Manager UI (browser)** — for verification only; this doc focuses on API-driven creation because the UI's TNP wizard has limitations on VDS-backed transport in NSX 9.
>
> The doc captures three ID-format gotchas. Replace any credential placeholder like `<your-password>` with values from your vault.

Creating a Transport Node Profile (TNP) on NSX 9 via API requires the **manager API** at `POST /api/v1/transport-node-profiles`. The policy API equivalent at `PATCH /policy/api/v1/infra/host-transport-node-profiles/{id}` returns **HTTP 405 Method not allowed**. This document captures the verified-working request body schema for VDS-backed transport, including three ID format gotchas that produced four sequential `HTTP 400` failures during a 2026-05-06 incident before the correct values were used:

1. `host_switch_id` is the **VDS UUID with spaces** (raw 16-byte format), e.g. `50 37 13 3f 96 e4 3e 93-1d f0 3b 0 1 d4 76 6a f8`
2. `ip_pool_id` is the **manager-API UUID** (e.g., `0b6b1dc5-...`), not the policy display name (`vcenter-cl01-tep-pool-01`)
3. `transport_zone_id` is the **manager-API UUID** (e.g., `3fbf2d31-...`), not the policy display name

After applying the correct schema with these IDs, TNP creation succeeded with HTTP 201, and applying the TNP to a 4-host cluster via TransportNodeCollection took **3.5 minutes** for all 4 hosts to transition from `Not Configured` → `success/success`.

This document includes the verified-working POST body, the lookup commands for each ID, the policy-API methods used to create the upstream TZs and IP Pool, and the TransportNodeCollection POST that ties the TNP to a vSphere cluster.

---

2. Endpoint Map

NSX 9 has both manager API (`/api/v1/...`) and policy API (`/policy/api/v1/infra/...`) namespaces. Different objects support different namespaces. The 2026-05-06 incident clarified these mappings:

Object	Verified Endpoint	Notes
Transport Zones	`PATCH /policy/api/v1/infra/sites/default/enforcement-points/default/transport-zones/{id}`	Policy API works; body uses `tz_type` (`OVERLAY_STANDARD` or `VLAN_BACKED`)
Uplink Profile	`POST /api/v1/host-switch-profiles`	Manager API; resource_type `UplinkHostSwitchProfile`
TEP IP Pool (create + subnet)	`PATCH /policy/api/v1/infra/ip-pools/{id}` then `PATCH /policy/api/v1/infra/ip-pools/{id}/ip-subnets/{subnet-id}`	Policy API; subnet body has `cidr` , `gateway_ip` , `allocation_ranges`
Transport Node Profile	**`POST /api/v1/transport-node-profiles`**	**Manager API only.** Policy API returns 405.
Transport Node Collection (TNC, the binding)	`POST /api/v1/transport-node-collections`	Manager API; body has `compute_collection_id` and `transport_node_profile_id`

---

3. Lookup Commands — Get the Right IDs

The four IDs needed for the TNP body must each be fetched from the right endpoint. **Do not reuse policy display names where manager UUIDs are expected.**

3.1 VDS `host_switch_id` (with spaces)

The VDS UUID is the 16-byte raw form formatted as `xx xx xx xx xx xx xx xx-xx xx xx xx xx xx xx xx`. Get it from any host that's already on the VDS:

```
 $ plink -ssh -batch -pw '<host-pw>' root@<esxi> 'esxcli network vswitch dvs vmware list | grep -A1 "VDS
ID"'
   VDS ID: 50 37 13 3f 96 e4 3e 93-1d f0 3b 01 d4 76 6a f8
```

Or via NSX after a host has been prepped at least once:

```
 $ curl -sk -u "admin:<pw>" "https://<nsx-mgr>/api/v1/transport-nodes/<tn-id>/state" \
    | python -c "import json,sys; d=json.load(sys.stdin); print(d['host_switch_states'][0]
['host_switch_id'])"
50 37 13 3f 96 e4 3e 93-1d f0 3b 01 d4 76 6a f8
```

3.2 Uplink Profile manager-API UUID

```
 $ curl -sk -u "admin:<pw>" "https://<nsx-mgr>/api/v1/host-switch-profiles" \
    | python -c "import json,sys
d = json.load(sys.stdin)
for p in d['results']:
    if p['resource_type']=='UplinkHostSwitchProfile' and 'vcenter-cl01' in p['display_name']:
        print(p['display_name'], '->', p['id'])"
vcenter-cl01-up-host01 -> 15e0debc-a169-4133-b7e3-c483236492a3
```

3.3 IP Pool manager-API UUID

The IP pool was created via the policy API (`PATCH /policy/api/v1/infra/ip-pools/vcenter-cl01-tep-pool-01`), but the TNP body needs the manager-API UUID, which differs.

```
 $ curl -sk -u "admin:<pw>" "https://<nsx-mgr>/api/v1/pools/ip-pools" \
    | python -c "import json,sys
d = json.load(sys.stdin)
for p in d['results']:
    if 'tep' in p['display_name'].lower():
        print(p['display_name'], '->', p['id'])"
vcenter-cl01-tep-pool-01 -> 0b6b1dc5-696b-4d21-b1da-72cad6390291
```

3.4 Transport Zone manager-API UUIDs

Same pattern: created via policy API but TNP body needs manager UUIDs.

```
 $ curl -sk -u "admin:<pw>" "https://<nsx-mgr>/api/v1/transport-zones" \
    | python -c "import json,sys
d = json.load(sys.stdin)
for tz in d['results']:
    if 'vcenter-cl01' in tz.get('display_name',''):
        print(tz['display_name'], 'type=', tz['transport_type'], 'id=', tz['id'])"
vcenter-cl01-tz-overlay01 type= OVERLAY id= 3fbf2d31-12a9-4781-8cd3-95b4bf472824
vcenter-cl01-tz-vlan01    type= VLAN    id= ce4eb145-59f2-4280-8480-a496366f8ff6
```

---

4. Verified-Working POST Body

```
 NSX="https://<nsx-mgr>"
AUTH='-u admin:<pw>'

cat > /tmp/tnp.json <<'EOF'
{
  "display_name": "vcenter-cl01-tnp-01",
  "host_switch_spec": {
    "resource_type": "StandardHostSwitchSpec",
    "host_switches": [
      {
        "host_switch_name": "vcenter-cl01-vds01-new",
        "host_switch_id": "50 37 13 3f 96 e4 3e 93-1d f0 3b 01 d4 76 6a f8",
        "host_switch_type": "VDS",
        "host_switch_mode": "STANDARD",
        "host_switch_profile_ids": [
          {
            "key": "UplinkHostSwitchProfile",
            "value": "15e0debc-a169-4133-b7e3-c483236492a3"
          }
        ],
        "uplinks": [
          {"vds_uplink_name": "dvUplink1", "uplink_name": "uplink-1"},
          {"vds_uplink_name": "dvUplink2", "uplink_name": "uplink-2"}
        ],
        "ip_assignment_spec": {
          "resource_type": "StaticIpPoolSpec",
          "ip_pool_id": "0b6b1dc5-696b-4d21-b1da-72cad6390291"
        },
        "transport_zone_endpoints": [
          {"transport_zone_id": "3fbf2d31-12a9-4781-8cd3-95b4bf472824"},
          {"transport_zone_id": "ce4eb145-59f2-4280-8480-a496366f8ff6"}
        ]
      }
    ]
  }
```

```
}
EOF

curl -sk --tlsv1.3 $AUTH -X POST "$NSX/api/v1/transport-node-profiles" \
  -H "Content-Type: application/json" -d @/tmp/tnp.json -w "\nHTTP %{http_code}\n"
```

Expected response: **HTTP 201**, full TNP object returned with auto-assigned `id` and `_create_time`.

---

5. The Four Failures Before This Worked (2026-05-06)

The verified-working body above evolved through four sequential errors. Each is documented because the error messages are not always self-explanatory.

5.1 Failure 1: Policy API endpoint — HTTP 405

```
 curl ... -X PATCH "https://<nsx-mgr>/policy/api/v1/infra/host-transport-node-profiles/vcenter-cl01-tnp-
01" -d '{...}'

{"module_name":"common-services","error_message":"Method is not allowed","error_code":282}
HTTP 405
```

Lesson: The policy API path is rejected for TNP creation. **Use the manager API** at `POST /api/v1/transport-node-profiles`.

5.2 Failure 2: IP pool by display name — HTTP 400 error 520033

```
 curl ... -X POST "https://<nsx-mgr>/api/v1/transport-node-profiles" -d '{
  "ip_assignment_spec": {
    "resource_type": "StaticIpPoolSpec",
    "ip_pool_id": "vcenter-cl01-tep-pool-01" # ← display name, NOT manager UUID
  }
  ...
}'

{"error_code":520033,"module_name":"policy",
 "error_message":"Invalid IPAddressPool path=[vcenter-cl01-tep-pool-01]."}
HTTP 400
```

Lesson: `ip_pool_id` requires the manager-API UUID from `/api/v1/pools/ip-pools`, not the policy display name. The policy API recognizes `vcenter-cl01-tep-pool-01` but the manager API does not.

5.3 Failure 3: Missing host_switch_id — HTTP 400 error 9529

After fixing the IP pool ID, next failure:

```
 {"error_code":9529,"module_name":"NsxSwitching service",
 "error_message":"Must provide host_switch_id for HostSwitch of type VDS. Node Id {0}."}
HTTP 400
```

Lesson: `host_switch_type: "VDS"` requires `host_switch_id` to be set explicitly. The VDS UUID format with embedded spaces (e.g., `50 37 13 3f 96 e4 3e 93-1d f0 3b 01 d4 76 6a f8`) is non-obvious and must come from the host or an existing TN, not from vCenter's MoRef ID.

5.4 Failure 4: TZ by display name — HTTP 400 error 9543

After adding host_switch_id:

```
 {"error_code":9543,"module_name":"NsxSwitching service",
 "error_message":"Cannot create HostSwitch {0} without HostSwitchMode and TransportZoneEndpoints."}
HTTP 400
```

The TZ endpoints WERE in the body, but as policy display names (`"transport_zone_id": "vcenter-cl01-tz-overlay01"`) which the manager API doesn't recognize. The error message complains that TZ endpoints are missing, when they're actually present-but-unrecognized.

Lesson: `transport_zone_id` requires the manager-API UUID from `/api/v1/transport-zones`, not the policy display name. **The error message is misleading** — it says "without TransportZoneEndpoints" when it really means "with unrecognized TransportZoneEndpoints".

5.5 Success on attempt 5

After replacing TZ display names with manager UUIDs (`3fbf2d31-...` and `ce4eb145-...`), the POST returned HTTP 201 and the TNP was created. Total time spent on the four schema corrections: ~10 minutes.

---

6. Applying the TNP — TransportNodeCollection

Once the TNP exists, bind it to a vSphere cluster via TransportNodeCollection (TNC). This triggers Fleet/EAM to push NSX prep to all hosts in the cluster.

6.1 Get current Compute Manager / cluster IDs

After CM register, find the compute collection ID (cluster) the TNP should be applied to. **Note that the prefix is the CM-UUID, which CHANGES if you re-register the CM** (e.g., during NSX rebuild).

```
 $ curl -sk -u "admin:<pw>" "https://<nsx-mgr>/api/v1/fabric/compute-collections" \
    | python -c "import json,sys
d=json.load(sys.stdin)
for cc in d['results']:
    if cc['origin_type']=='VC_Cluster':
        print(cc['display_name'], '->', cc['external_id'])"
vcenter-cl01 -> 259b8edb-3b54-496f-bfb4-c85c145cd9bb:domain-c10
```

The format is `<CM-UUID>:domain-c<NN>`. The `<CM-UUID>` portion is the new CM ID created during the most recent NSX register. **An old prefix from a prior NSX session will NOT work** — that was a 2026-05-06 failure mode where a stale `5b94798e-...:domain-c10` from a prior CM registration was tried first and rejected with HTTP 404.

6.2 POST TNC body

```
 cat > /tmp/tnc.json <<'EOF'
{
  "display_name": "vcenter-cl01-tnc",
  "description": "Apply TNP to vcenter-cl01",
  "resource_type": "TransportNodeCollection",
  "compute_collection_id": "259b8edb-3b54-496f-bfb4-c85c145cd9bb:domain-c10",
  "transport_node_profile_id": "<TNP-UUID-from-section-4-response>"
}
EOF
```

```
curl -sk -u "admin:<pw>" -X POST "https://<nsx-mgr>/api/v1/transport-node-collections" \
  -H "Content-Type: application/json" -d @/tmp/tnc.json -w "\nHTTP %{http_code}\n"
```

Expected: HTTP 201 with TNC object containing auto-assigned `id`.

6.3 Verify hosts transition to success

After TNC POST, all hosts in the cluster begin NSX prep. Watch their state via:

```
 $ curl -sk -u "admin:<pw>" "https://<nsx-mgr>/api/v1/transport-nodes/state" \
    | python -c "import json,sys
d=json.load(sys.stdin)
for tn in d['results']:
    print(tn.get('state'), tn.get('node_deployment_state',{}).get('state'))"
```

Expected progression (~3-5 min for a 4-host cluster):

```
 pending  in_progress    ← all 4 like this for 2-3 min
in_progress  in_progress  ← brief transition
success  success         ← all 4 within ~3.5 min on a healthy cluster
```

Verified 2026-05-06: Following this exact schema, all 4 hosts went from `Not Configured` → `success/success` in **3 minutes 39 seconds**.

---

7. Building the Upstream Objects (TZ, Uplink Profile, IP Pool)

The TNP references upstream objects. Here are the verified-working POST/PATCH bodies for each.

7.1 Transport Zones (Policy API)

```
 # Overlay TZ
curl -sk -u "admin:<pw>" -X PATCH \
  "https://<nsx-mgr>/policy/api/v1/infra/sites/default/enforcement-points/default/transport-zones/vcenter-
cl01-tz-overlay01" \
  -H "Content-Type: application/json" -d '{
    "display_name": "vcenter-cl01-tz-overlay01",
    "tz_type": "OVERLAY_STANDARD"
  }'
# Expected: HTTP 200

# VLAN TZ
curl -sk -u "admin:<pw>" -X PATCH \
  "https://<nsx-mgr>/policy/api/v1/infra/sites/default/enforcement-points/default/transport-zones/vcenter-
cl01-tz-vlan01" \
  -H "Content-Type: application/json" -d '{
    "display_name": "vcenter-cl01-tz-vlan01",
    "tz_type": "VLAN_BACKED"
  }'
# Expected: HTTP 200
```

7.2 Uplink Profile (Manager API)

```
 # Note: MTU intentionally NOT set. For VDS-backed transport, MTU is owned by the
# vSphere VDS object, not the NSX Uplink Profile. Setting MTU here returns Error 9531.

curl -sk -u "admin:<pw>" -X POST "https://<nsx-mgr>/api/v1/host-switch-profiles" \
```

```
  -H "Content-Type: application/json" -d '{
    "display_name": "vcenter-cl01-up-host01",
    "resource_type": "UplinkHostSwitchProfile",
    "lags": [],
    "teaming": {
      "policy": "LOADBALANCE_SRCID",
      "active_list": [
        {"uplink_name": "uplink-1", "uplink_type": "PNIC"},
        {"uplink_name": "uplink-2", "uplink_type": "PNIC"}
      ],
      "standby_list": []
    },
    "transport_vlan": 0,
    "named_teamings": []
  }'
# Expected: HTTP 201
```

7.3 TEP IP Pool (Policy API, two-step)

```
 # Step 1: Create the pool
curl -sk -u "admin:<pw>" -X PATCH \
  "https://<nsx-mgr>/policy/api/v1/infra/ip-pools/vcenter-cl01-tep-pool-01" \
  -H "Content-Type: application/json" -d '{
    "display_name": "vcenter-cl01-tep-pool-01",
    "resource_type": "IpAddressPool"
  }'
# Expected: HTTP 200

# Step 2: Add subnet with allocation range
# IMPORTANT: size pool for (uplinks per host) x (number of hosts) + headroom.
# 4 hosts x 2 uplinks = 8 IPs minimum. Recommend 16-20 for headroom.

curl -sk -u "admin:<pw>" -X PATCH \
  "https://<nsx-mgr>/policy/api/v1/infra/ip-pools/vcenter-cl01-tep-pool-01/ip-subnets/tep-subnet" \
  -H "Content-Type: application/json" -d '{
    "resource_type": "IpAddressPoolStaticSubnet",
    "display_name": "tep-subnet",
    "cidr": "192.168.13.0/24",
    "gateway_ip": "192.168.13.1",
    "allocation_ranges": [
      {"start": "192.168.13.101", "end": "192.168.13.120"}
    ]
  }'
# Expected: HTTP 200
```

---

8. Lessons Learned

#	Lesson
L-1	TNP creation is **manager API only**. The policy API equivalent at `/policy/api/v1/infra/host-transport-node-profiles/{id}` returns HTTP 405.
L-2	TZ and IP Pool **creation** uses policy API (PATCH), but **references in the TNP body** use manager-API UUIDs. The two namespaces store the same logical object under different IDs.
L-3	`host_switch_id` for VDS-backed transport is the raw 16-byte UUID with spaces, e.g. `50 37 13 3f 96 e4 3e 93-1d f0 3b 01 d4 76 6a f8`. Get it from `esxcli network vswitch dvs vmware list` on a host or from an existing TN's state. **NOT the vCenter MoRef ID.**
L-4	Error 9543 "Cannot create HostSwitch without TransportZoneEndpoints" is misleading when TZs ARE in the body but as policy display names instead of manager UUIDs. The validator silently treats unrecognized IDs as missing.

#	Lesson
L-5	The CM-UUID prefix in `compute_collection_id` (`<CM-UUID>:domain-c<NN>`) changes every time the Compute Manager is re-registered. After NSX rebuild, always re-fetch via `/api/v1/fabric/compute-collections` — old prefixes from prior sessions return HTTP 404.
L-6	Do not set MTU in the Uplink Profile when using VDS-backed transport. MTU is owned by the vSphere VDS object. Setting MTU in the Uplink Profile returns Error 9531.
L-7	TEP pool sizing: `(uplinks_per_host &times; hosts) + headroom`. For 2-uplink hosts in NSX 9 multi-VTEP default, 4 hosts need 8 IPs minimum. Recommend 16-20 for headroom. The 2026-04-25 deploy hit pool-exhausted with only 4 IPs.
L-8	Once TNP + TNC binding are correct, all hosts in a 4-host cluster transition `Not Configured` → `success/success` in **~3-5 minutes** on a healthy cluster with VIBs already installed. If transition takes > 10 min, something is wrong — check `/api/v1/transport-nodes/{id}/state` for `failure_message`.

---

9. Cross-References

- `NSX-Manager-Cold-Start-Troubleshooting-Report.md` Addendum D — the broader 2026-05-06 NSX rebuild context where this schema was discovered
- `VCF9-Host-vCenter-Rebuild-Plan.md` Phase 4 P4.9 — the user-authored procedure that prescribes the upstream objects (TZ, Uplink Profile, IP Pool, TNP)
- `vCenter-CertificateManagement-Service-90348-Recovery.md` — sister document for the CM register precondition
- `NSX9-OVA-PowerCLI-ImportVApp-vs-ovftool.md` — sister document for the NSX appliance deploy that precedes TNP creation
- VMware NSX 9 API documentation — for the upstream schema definition

---

10. Document Information

Field	Value
Document Title	NSX 9 Transport Node Profile — Schema Reference
Prepared By	Virtual Control LLC
Date	May 6, 2026
Document Version	1.0
Classification	Internal — API Reference
VCF Version	9.0.1
NSX Build Tested	9.0.1.0 build 24952114
Verification	TNP creation: 5 attempts (4 fail, 1 success). TNC apply: all 4 hosts state=success in 3m39s.
Endpoint Tested	`https://192.168.1.71/api/v1/transport-node-profiles`
